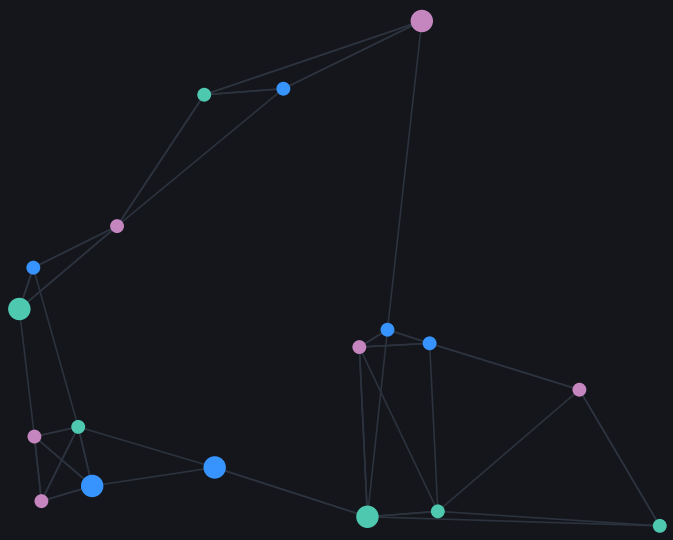


COMPUTER NETWORKS LAB

Socket Programming & Error Detection

Eight lab assignments in Python, each with dark-theme code, an explanation beside every line, sample output from real runs, and a viva guide that goes from the basics to the code.

py tcp_server.py × Python 3

```
7  server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
8  server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
9  server.bind((HOST, PORT))
10 server.listen(1)
11 print("Server listening on", HOST, "port", PORT)
12
13 conn, addr = server.accept()
```

INSIDE

01 TCP Client-Server
02 TCP Echo
03 TCP Text File Transfer
04 UDP Echo
05 UDP Timestamp
06 Checksum
07 CRC
08 TCP Image Transfer

Python 3 · standard library only · every program tested end to end on localhost

Contents

Before you start . . . 3

01 Simple TCP Client-Server Application . . . 5
- Code with line-by-line explanation . . . 5
- Viva guide . . . 7

02 TCP Echo Server-Client Application . . . 9
- Code with line-by-line explanation . . . 9
- Viva guide . . . 11

03 TCP Text File Transfer Application . . . 13
- Code with line-by-line explanation . . . 13
- Viva guide . . . 15

04 UDP Echo Server-Client Application . . . 17
- Code with line-by-line explanation . . . 17
- Viva guide . . . 19

05 UDP Timestamp Application . . . 21
- Code with line-by-line explanation . . . 21
- Viva guide . . . 23

06 Checksum Error Detection . . . 24
- Code with line-by-line explanation . . . 25
- Viva guide . . . 26

07 Cyclic Redundancy Check (CRC) . . . 28
- Code with line-by-line explanation . . . 29
- Viva guide . . . 30

08 TCP Image File Transfer Application . . . 32
- Code with line-by-line explanation . . . 32
- Viva guide . . . 34

Before you start

Each assignment follows the same pattern: what the program does and how the two sides talk, the full code in a dark editor panel with an explanation **beside every line**, how to run it with real output, and a viva guide. The explanation on the right always belongs to the code line on its left.

Running the programs

1. Install **Python 3.8 or newer**. Nothing else is needed: every program uses only the standard library (`socket`, `os`, `datetime`).
2. Put each assignment's files in one folder. For the file transfers, keep `sample.txt` or `photo.jpg` next to the client.
3. Open **two terminals** in that folder. Start the **server first** in one, then the **client** in the other. Assignments 6 and 7 are single programs.
4. Use `python file.py` on Windows (or `py file.py`); on macOS and Linux use `python3 file.py`.
5. Stop a server with **Ctrl+C**. On Windows, if Ctrl+C is ignored while the server waits, press **Ctrl+Break** or close that terminal.

Assignments at a glance

#	Assignment	Protocol	Port	Files
1	Simple TCP client-server	TCP	5000	`tcp_server.py, tcp_client.py`
2	TCP echo	TCP	6000	`echo_server.py, echo_client.py`
3	TCP text file transfer	TCP	7000	`file_server.py, file_client.py, sample.txt`
4	UDP echo	UDP	8000	`udp_echo_server.py, udp_echo_client.py`
5	UDP timestamp	UDP	9000	`time_server.py, time_client.py`
6	Checksum	—	—	`checksum.py`
7	CRC	—	—	`crc.py`
8	TCP image file transfer	TCP	7500	`image_server.py, image_client.py, photo.jpg`

Socket calls you will meet

Call	Used by	What it does
`socket(AF_INET, SOCK_STREAM)`	TCP, both	Create a TCP socket over IPv4.
`socket(AF_INET, SOCK_DGRAM)`	UDP, both	Create a UDP socket over IPv4.
`bind((host, port))`	Server	Attach the socket to an IP address and port.
`listen(n)`	TCP server	Start accepting connections; n = backlog queue size.
`accept()`	TCP server	Wait for a client; returns (new socket, client address).
`connect((host, port))`	TCP client	Open a connection (three-way handshake).
`sendall(data)`	TCP	Send all the bytes; keeps going until done.
`recv(n)`	TCP	Receive up to n bytes; b"" means the peer closed.
`sendto(data, addr)`	UDP	Send one datagram to addr.
`recvfrom(n)`	UDP	Receive one datagram; returns (data, sender address).
`settimeout(sec)`	Both	Give up waiting after sec seconds (raises socket.timeout).
`shutdown(SHUT_WR)`	TCP	Half-close: stop sending, still able to receive.
`setsockopt(..., SO_REUSEADDR, 1)`	Server	Allow re-binding a port that was just used.
`close()`	Both	Release the socket (TCP also closes the connection).

TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented (three-way handshake)	Connectionless
Reliability	Lost data is retransmitted	No retransmission; data may be lost
Order	Bytes arrive in the order sent	Datagrams may arrive out of order
Unit of data	Byte stream (no message boundaries)	Datagram (boundaries kept)
Header size	20 to 60 bytes	8 bytes
Speed / overhead	Slower, more overhead	Faster, very light
Python socket type	`SOCK_STREAM`	`SOCK_DGRAM`
Typical uses	Web (HTTP/HTTPS), email, file transfer, SSH	DNS, DHCP, streaming, VoIP, games
In this guide	Assignments 1, 2, 3, 8	Assignments 4, 5

When something goes wrong

Error	Usual cause	Fix
ConnectionRefusedError	Server isn't running, or HOST/PORT don't match.	Start the server first; check both files use the same port.
Address already in use (WinError 10048)	An old server still holds the port.	Stop it with Ctrl+C or close its terminal, then retry.
FileNotFoundError	sample.txt / photo.jpg isn't in the client's folder.	Put the file next to the client, or change FILE in the code.
UDP: "No reply" / "did not respond"	The UDP server isn't running or the port differs.	Start the server; check PORT on both sides.
Program seems frozen	It is blocked in `accept()` / `recv()` waiting for the other side.	Normal. Start the other program, or stop with Ctrl+C.

Running across two computers

On the server, set `HOST = "0.0.0.0"` (listen on every network interface). On the client, set `HOST` to the server computer's IP address, found with `ipconfig` (Windows) or `ip a` (Linux). Both machines must be on the same network, and the server's firewall must allow the port.

ASSIGNMENT 01

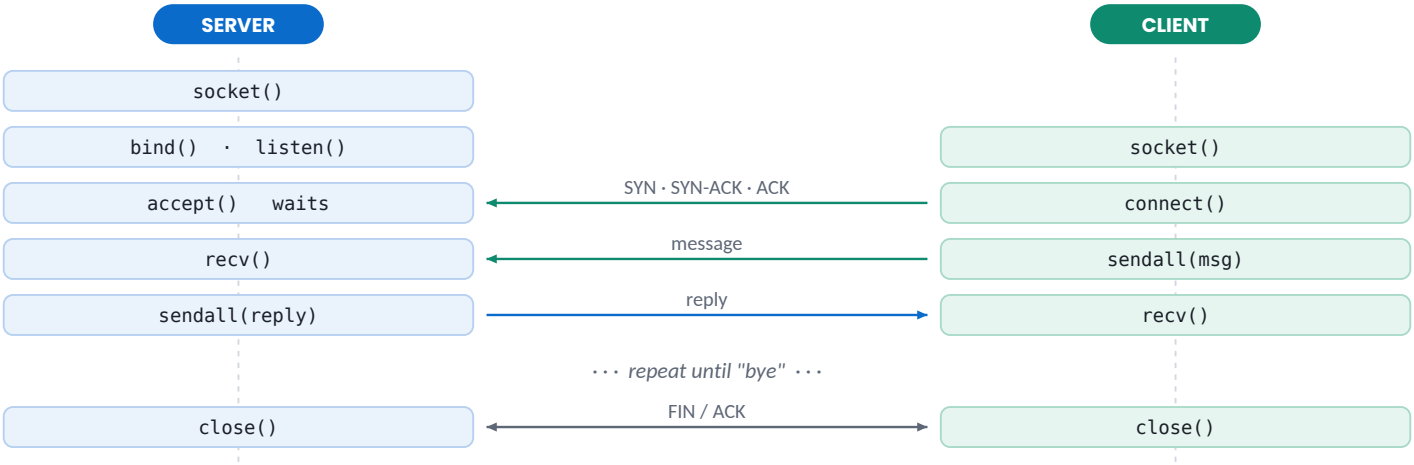
Simple TCP Client-Server Application

Aim. Establish a TCP connection between a client and a server and exchange text messages in both directions.

TCP | Port 5000 | SOCK_STREAM | tcp_server.py | tcp_client.py

How it works

The server creates a socket, binds it to 127.0.0.1:5000 and waits in `accept()`. The client's `connect()` completes TCP's three-way handshake, after which both sides take turns: one sends, the other receives and replies. Typing **bye** on either side ends the chat and both sockets are closed.



Code with line-by-line explanation

py tcp_server.py × Python 3

		WHAT EACH LINE DOES
1	`# tcp_server.py - Simple TCP chat server`	Comment: names the file and its purpose. Python ignores everything after #.
2	`import socket`	Imports Python's built-in `socket` module, a wrapper around the operating system's socket API. Nothing to install.
3		
4	`HOST = "127.0.0.1"`	IP address the server listens on. `127.0.0.1` is **localhost** (loopback): this same computer.
5	`PORT = 5000`	**Port number** that identifies this program on the machine. Ports above 1023 need no admin rights.
6		
7	`server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)`	Creates the server socket. `AF_INET` = IPv4 addresses; `SOCK_STREAM` = **TCP** (reliable byte stream).
8	`server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)`	Lets the port be reused at once after the server stops, avoiding the "Address already in use" error on re-run.
9	`server.bind((HOST, PORT))`	**Binds** the socket to (IP, port) so the OS hands connections for port 5000 to this program. Takes one tuple.
10	`server.listen(1)`	Switches to **listening** mode. `1` is the backlog: how many pending connections may wait in the queue.
11	`print("Server listening on", HOST, "port", PORT)`	Status message so we know the server is ready.
12		
13	`conn, addr = server.accept()`	**Blocks** (waits) until a client connects. Returns `conn`, a new socket for this client, and `addr`, the client's (IP, port).
14	`print("Connected to client:", addr)`	Shows the client's address, e.g. ('127.0.0.1', 58722). That port was picked automatically by the client's OS.
15		
16	`while True:`	Infinite loop: the chat goes on until someone types "bye".

py tcp_server.py × Python 3

		WHAT EACH LINE DOES
17	`    data = conn.recv(1024).decode()`	Receives up to 1024 bytes from the client; `decode()` turns those bytes into a string (UTF-8).
18	`    if not data or data.lower() == "bye":`	Empty data means the client closed the connection; "bye" means it wants to stop. `lower()` makes the test case-insensitive.
19	`        print("Client ended the chat.")`	Tells the server user that the chat is over.
20	`        break`	`break` leaves the while loop.
21	`    print("Client:", data)`	Displays the client's message.
22	`    reply = input("Server: ")`	Waits for the server user to type a reply on the keyboard.
23	`    conn.sendall(reply.encode())`	`encode()` turns the reply into bytes and `sendall()` sends every byte of it to the client.
24	`    if reply.lower() == "bye":`	If the server typed "bye"...
25	`        break`	...end the chat from the server side.
26		
27	`conn.close()`	Closes the connection socket for this client (TCP sends FIN to end the connection).
28	`server.close()`	Closes the listening socket and frees port 5000.

Python 3 · UTF-8 · LF · Spaces: 4 28 lines

py tcp_client.py × Python 3

		WHAT EACH LINE DOES
1	`# tcp_client.py - Simple TCP chat client`	Comment: file name and purpose.
2	`import socket`	Imports the `socket` module.
3		
4	`HOST = "127.0.0.1"`	Address of the server to connect to. Must match the server's HOST (use the server's real IP across two computers).
5	`PORT = 5000`	Must be the same port the server is listening on.
6		
7	`client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)`	Creates a TCP/IPv4 socket. The client never calls `bind()`: the OS picks a free (ephemeral) port for it.
8	`client.connect((HOST, PORT))`	Starts the **three-way handshake** (SYN, SYN-ACK, ACK). Raises ConnectionRefusedError if the server isn't running.
9	`print("Connected to server at", HOST, "port", PORT)`	Confirms the connection succeeded.
10		
11	`while True:`	Chat loop: runs until "bye".
12	`    msg = input("Client: ")`	Reads a line typed by the client user.
13	`    client.sendall(msg.encode())`	Converts the text to bytes and sends it. Sockets carry bytes, not strings.
14	`    if msg.lower() == "bye":`	If the client typed "bye"...
15	`        break`	...leave the loop. The "bye" was already sent, so the server knows too.
16	`    reply = client.recv(1024).decode()`	Waits for the server's reply (up to 1024 bytes) and decodes it into a string.
17	`    if not reply or reply.lower() == "bye":`	Empty reply = server closed the connection; "bye" = server ended the chat.
18	`        print("Server ended the chat.")`	Tells the user the server ended the chat.
19	`        break`	Leaves the loop.
20	`    print("Server:", reply)`	Displays the server's reply.
21		
22	`client.close()`	Closes the socket, which ends the TCP connection.

Python 3 · UTF-8 · LF · Spaces: 4 22 lines

Run it

TERMINAL 1
```
python tcp_server.py
```
TERMINAL 2
```
python tcp_client.py
```

Start the server first. Messages alternate: the client speaks, then the server replies. Type bye to finish.

Sample output (text in **amber** is what you type)

Terminal 1 - server
```
$ python tcp_server.py
Server listening on 127.0.0.1 port 5000
Connected to client: ('127.0.0.1', 58722)
Client: Hello server
Server: Hello client!
Client: How are you?
Server: All good. Bye for now.
Client ended the chat.
```

Terminal 2 - client
```
$ python tcp_client.py
Connected to server at 127.0.0.1 port 5000
Client: Hello server
Server: Hello client!
Client: How are you?
Server: All good. Bye for now.
Client: bye
```

Viva guide · Assignment 1

Questions run from definitions to concepts to this program's code. Answers are written the way you would say them in a viva.

LEVEL 1 · BASICS Definitions you should know cold

Q1 What is a computer network?

A set of devices (nodes) joined by communication links so they can share data and resources. A college lab LAN and the Internet are both networks.

Q2 What is a protocol?

An agreed set of rules for communication: the format of messages, their order, and what each side does on sending or receiving them. TCP, UDP, IP and HTTP are protocols.

Q3 What is an IP address?

A logical address that identifies a host on a network. IPv4 uses 32 bits written as four numbers (e.g. 192.168.1.10); IPv6 uses 128 bits.

Q4 What is a port?

A 16-bit number (0 to 65535) that identifies a particular process or service on a host. The IP address delivers data to the right **computer**; the port delivers it to the right **program**. Well-known ports are 0-1023 (HTTP 80, HTTPS 443, SSH 22, DNS 53), registered ports 1024-49151, and dynamic or ephemeral ports 49152-65535.

Q5 What is a socket?

One endpoint of a two-way communication link, identified by IP address + port number + protocol. In code, it is the object the operating system gives us to send and receive data over the network.

Q6 Explain the client-server model.

The server starts first and waits passively at a known address and port. The client initiates contact, sends requests and receives responses. A web browser (client) and a web server are the classic example.

Q7 What is localhost / 127.0.0.1?

The loopback address. Packets sent to it never leave the computer, so the client and server can be tested on one machine.

LEVEL 2 · CONCEPTS How and why it works

Q8 What is TCP?

Transmission Control Protocol: a **connection-oriented, reliable** transport-layer protocol. It delivers a stream of bytes in order, retransmits lost segments, and provides flow control and congestion control.

Q9 Explain the three-way handshake.

1) The client sends SYN with its initial sequence number. 2) The server replies SYN-ACK. 3) The client sends ACK. The connection is now established. In our code `connect()` starts it and `accept()` returns once it completes.

Q10 How is a TCP connection closed?

Each direction is closed separately with FIN and ACK, usually four segments in total (FIN, ACK, FIN, ACK). Calling `close()` starts this.

Q11 At which layer do sockets work?

The socket API sits between the application layer (our program) and the transport layer (TCP/UDP), which the operating system implements.

Q12 Why does the server call bind() but the client doesn't?

Clients must know where to find the server, so the server needs a fixed, known port. The client's port can be anything, so the OS assigns a free ephemeral port automatically during `connect()`.

Q13 Is this chat full-duplex or half-duplex?

Half-duplex in practice: each side alternates between sending and waiting, because `input()` and `recv()` both block. A full-duplex chat would use two threads, one for sending and one for receiving.

LEVEL 3 · CODE WALKTHROUGH Questions about this program

Q14 What do AF_INET and SOCK_STREAM mean?

`AF_INET` is the IPv4 address family (`AF_INET6` is IPv6). `SOCK_STREAM` is a stream socket, which means TCP. (`SOCK_DGRAM` would mean UDP.)

Q15 Why is (HOST, PORT) passed with double brackets?

`bind()` and `connect()` take a single argument: an address tuple. The inner brackets build that tuple.

Q16 What does listen(1) do, and what is the 1?

It marks the socket as passive, ready to accept connections. The 1 is the **backlog**: how many completed connections may wait in the queue before the server calls `accept()`.

Q17 Why does accept() return a new socket?

The listening socket keeps waiting for new clients, while `conn` is a separate socket dedicated to this one client. That separation is what lets a server handle many clients.

Q18 Why do we use encode() and decode()?

Sockets carry bytes only. `encode()` converts a string to bytes (UTF-8 by default) before sending; `decode()` converts received bytes back to a string.

Q19 Does recv(1024) always return 1024 bytes?

No. 1024 is the maximum. It returns whatever has arrived (at least 1 byte) and returns empty bytes `b""` when the other side has closed the connection.

Q20 What is SO_REUSEADDR for?

After a server stops, its port can stay in the TIME_WAIT state for a while. `SO_REUSEADDR` lets us bind to it again immediately instead of getting "Address already in use".

Q21 What happens if the client runs before the server?

`connect()` fails with ConnectionRefusedError because nothing is listening on port 5000 yet.

ASSIGNMENT 02

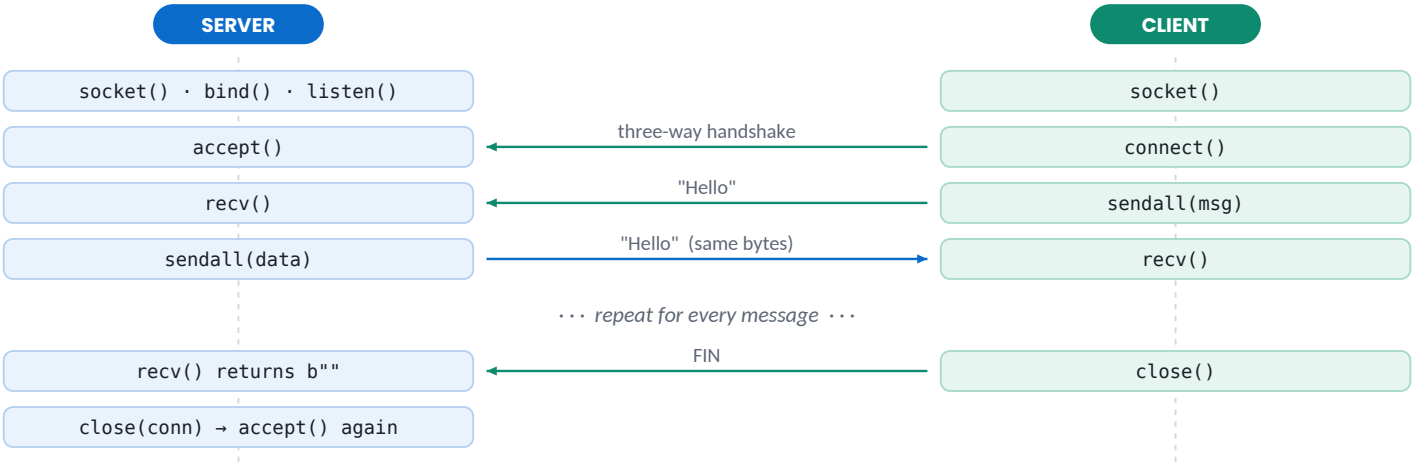
TCP Echo Server-Client Application

Aim. Build a TCP server that sends every message it receives back to the client, unchanged.

TCP | Port 6000 | Iterative server | echo_server.py | echo_client.py

How it works

The server runs forever. For each client it accepts, an inner loop reads bytes with `recv()` and immediately writes the same bytes back with `sendall()`. When the client closes, `recv()` returns empty bytes, the server closes that connection and goes back to `accept()` for the next client. Ctrl+C stops the server cleanly.



Code with line-by-line explanation

py echo_server.py × Python 3

		WHAT EACH LINE DOES
1	`# echo_server.py - TCP echo server`	Comment: file name and purpose.
2	`import socket`	Imports the `socket` module.
3		
4	`HOST = "127.0.0.1"`	Server IP: localhost.
5	`PORT = 6000`	Port 6000 for our echo service. (The official echo service uses port 7, which needs admin rights.)
6		
7	`server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)`	Creates a **TCP** (`SOCK_STREAM`) socket over IPv4 (`AF_INET`).
8	`server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)`	Allows an immediate restart on the same port (`SO_REUSEADDR`).
9	`server.bind((HOST, PORT))`	Attaches the socket to 127.0.0.1:6000.
10	`server.listen(5)`	Starts listening; up to 5 connection requests can wait in the backlog queue.
11	`print("Echo server running on port", PORT)`	Status message.
12		
13	`try:`	`try` block, so pressing Ctrl+C is handled cleanly instead of crashing.
14	`    while True:`	Outer loop: keeps the server alive to serve one client after another.
15	`        conn, addr = server.accept()`	Waits for a client; returns a new socket `conn` for it and its address `addr`.
16	`        print("Client connected:", addr)`	Logs which client connected.
17	`        while True:`	Inner loop: keeps echoing for this client until it disconnects.
18	`            data = conn.recv(1024)`	Receives up to 1024 bytes. Kept as **bytes** because they go back unchanged.
19	`            if not data:`	`recv()` returns empty bytes `b""` when the client closes its side...

py echo_server.py × Python 3

```
                break
            print("Received:", data.decode())
            conn.sendall(data)

        print("Client disconnected:", addr)
        conn.close()

except KeyboardInterrupt:
    print("\nServer shutting down.")

finally:
    server.close()
```

Python 3 · UTF-8 · LF · Spaces: 4 28 lines

WHAT EACH LINE DOES
...so leave the inner loop.
Logs the message (decoded to text only for display).
The echo: sends exactly the same bytes straight back to the client.
Logs the disconnection.
Closes this client's socket; the outer loop goes back to `accept()`.
Runs when the user presses Ctrl+C in the server terminal.
Shutdown message. The `\n` starts a new line after the `^C` the terminal prints.
`finally` runs however the `try` block ended...
...so the listening socket is always closed and the port freed.

py echo_client.py × Python 3

```
# echo_client.py - TCP echo client
import socket

HOST = "127.0.0.1"
PORT = 6000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))
print("Connected to echo server.")

while True:
    msg = input("Enter message (exit to quit): ")
    if msg.lower() == "exit":
        break
    if not msg:
        continue
    client.sendall(msg.encode())
    echo = client.recv(1024).decode()
    print("Echo from server:", echo)

client.close()
```

Python 3 · UTF-8 · LF · Spaces: 4 21 lines

WHAT EACH LINE DOES
Comment: file name and purpose.
Imports the `socket` module.
Server address (localhost).
Must match the server's port, 6000.
Creates a TCP client socket.
Connects to the echo server (three-way handshake).
Confirms the connection.
Loop: keep sending messages until the user types "exit".
Reads a message from the keyboard.
Checks for the quit command (case-insensitive).
Leaves the loop.
Empty input would send 0 bytes, and the client would then wait forever for a reply...
...so skip it: `continue` jumps straight to the next loop iteration.
Encodes the message to bytes and sends all of it.
Waits for the echoed bytes and decodes them back to a string.
Prints the echo; it should be identical to what was typed.
Closing the socket makes the server's `recv()` return `b""`, so the server knows we left.

Run it

TERMINAL 1

```
python echo_server.py
```

TERMINAL 2

```
python echo_client.py
```

Type messages in the client; each comes straight back. Type exit to leave, then Ctrl+C in the server terminal.

Sample output (text in **amber** is what you type)

Terminal 1 - server

```
$ python echo_server.py
Echo server running on port 6000
Client connected: ('127.0.0.1', 48244)
Received: Hello echo server
Received: Networks lab
Received: TCP is reliable
Client disconnected: ('127.0.0.1', 48244)
^C
Server shutting down.
```

Terminal 2 - client

```
$ python echo_client.py
Connected to echo server.
Enter message (exit to quit): Hello echo server
Echo from server: Hello echo server
Enter message (exit to quit): Networks lab
Echo from server: Networks lab
Enter message (exit to quit): TCP is reliable
Echo from server: TCP is reliable
Enter message (exit to quit): exit
```

Viva guide · Assignment 2

Questions run from definitions to concepts to this program's code. Answers are written the way you would say them in a viva.

LEVEL 1 · BASICS Definitions you should know cold

Q1 What is an echo server?

A server that returns exactly what it receives. It is used to test connectivity, measure round-trip time and debug clients. The standard echo service (RFC 862) uses port 7.

Q2 What is round-trip time (RTT)?

The time for a message to reach the server plus the time for the reply to come back. An echo server is the simplest way to measure it.

Q3 How is ping different from our echo server?

`ping` uses ICMP Echo Request/Reply at the network layer and needs no server program. Our echo server works at the application layer over TCP and needs a program listening on a port.

Q4 What is an iterative server? What is a concurrent server?

An iterative server handles one client at a time; others wait in the backlog (ours is iterative). A concurrent server handles many clients at once, using threads, processes, or `select`/`asyncio`.

LEVEL 2 · CONCEPTS How and why it works

Q5 Does TCP preserve message boundaries?

No. TCP is a **byte stream**: two sends can arrive in one `recv()`, and one send can be split across several. Real protocols add delimiters (e.g. newline) or a length prefix to mark messages.

Q6 What is blocking I/O?

A blocking call pauses the program until it can finish. `accept()` blocks until a client connects, `recv()` until data arrives, `input()` until the user presses Enter.

Q7 What is the TIME_WAIT state?

After a connection closes, the side that closed first waits about twice the maximum segment lifetime so late packets from the old connection can't confuse a new one. It is why a restarted server may see "Address already in use".

Q8 send() vs sendall()?

`send()` may transmit only part of the data and returns how many bytes it sent. `sendall()` keeps sending until every byte has gone (or raises an error), so we don't have to loop ourselves.

LEVEL 3 · CODE WALKTHROUGH Questions about this program

Q9 Why are there two while loops in the server?

The outer loop accepts clients one after another, forever. The inner loop keeps echoing for the current client until that client disconnects.

Q10 How does the server know the client has left?

When the client closes its socket, TCP sends FIN, and the server's `recv()` returns empty bytes `b""`. The `if not data: break` line checks for exactly that.

Q11 Why isn't the data decoded before it is sent back?

The echo must return the exact bytes received. Decoding is done only to print the message; sending the original bytes avoids any change from re-encoding.

Q12 Why listen(5) here, when assignment 1 used listen(1)?

This server is meant to serve many clients over its lifetime, so a larger backlog lets several connection requests queue while one client is being served.

Q13 Why does the client skip empty input?

`sendall(b"")` sends nothing, so the server never replies, and the client's `recv()` would block forever.

Q14 What do try, except and finally do here?

Ctrl+C raises KeyboardInterrupt. The `except` block prints a message instead of a traceback, and `finally` always closes the listening socket so the port is released.

Q15 How would you serve several clients at the same time?

Start a new thread for each accepted connection, e.g. `threading.Thread(target=handle, args=(conn,)).start()`, or use `socketserver.ThreadingTCPServer`, `select`, or `asyncio`.

ASSIGNMENT 03

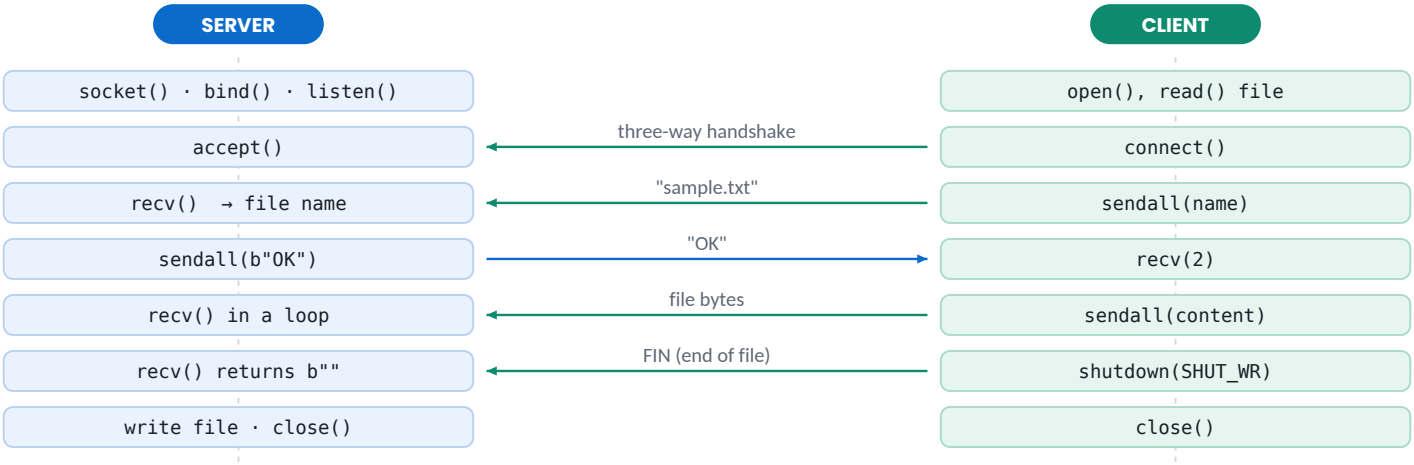
TCP Text File Transfer Application

Aim. Send a text file from the client to the server over TCP and store it on the server side.

TCP | Port 7000 | Text mode | file_server.py | file_client.py

How it works

A tiny protocol is used so the two programs stay in step: the client sends the **file name**, waits for the server's **OK**, then sends the **file content** and half-closes the connection. The server collects bytes until `recv()` returns empty, decodes them as UTF-8 text and saves them as received_<name>.



Code with line-by-line explanation

py file_server.py × Python 3

	Code	WHAT EACH LINE DOES
1	`# file_server.py - receives a text file over TCP`	Comment: file name and purpose.
2	`import socket`	Imports the `socket` module.
3	`import os`	Imports `os`, used for `os.path.basename()` to clean up the file name.
4		
5	`HOST = "127.0.0.1"`	Localhost.
6	`PORT = 7000`	Port 7000 for the file service.
7		
8	`server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)`	**TCP** socket: a file must arrive complete and in order, which TCP guarantees.
9	`server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)`	Allows a quick restart on the same port.
10	`server.bind((HOST, PORT))`	Binds to 127.0.0.1:7000.
11	`server.listen(1)`	Listens for one client.
12	`print("File server waiting on port", PORT)`	Status message.
13		
14	`conn, addr = server.accept()`	Waits for the client to connect.
15	`print("Connected to", addr)`	Shows the client's address.
16		
17	`filename = conn.recv(1024).decode()`	The client's first message is the **file name**; decode it to a string.
18	`conn.sendall(b"OK")`	Sends a 2-byte acknowledgement. The client waits for it, so the name and the file content never merge into one `recv()`. `b"OK"` is a bytes literal.
19		
20	`data = b""`	Empty bytes object that will collect the file's contents.
21	`while True:`	Keep reading until the client finishes sending.

py file_server.py × Python 3

		WHAT EACH LINE DOES
22	`    chunk = conn.recv(1024)`	Reads the next **chunk** of up to 1024 bytes. A file can be bigger than one `recv()`.
23	`    if not chunk:`	`b""` means the client called `shutdown()`: end of file reached.
24	`        break`	Stop receiving.
25	`    data += chunk`	Appends the chunk to the data collected so far.
26		
27	`save_as = "received_" + os.path.basename(filename)`	Output name, e.g. received_sample.txt. `basename()` strips any folder path the client sent, so nothing is written outside this folder.
28	`with open(save_as, "w", encoding="utf-8") as f:`	Opens the output file in **text-write** mode ("w") as UTF-8. `with` closes it automatically.
29	`    f.write(data.decode("utf-8"))`	Decodes all the bytes to text in one go (decoding chunk by chunk could split a multi-byte character) and writes it.
30		
31	`print("Saved", save_as, "-", len(data), "bytes")`	Reports the saved file name and its size in bytes.
32	`conn.close()`	Closes the client connection.
33	`server.close()`	Closes the listening socket.

Python 3 · UTF-8 · LF · Spaces: 4 33 lines

py file_client.py × Python 3

		WHAT EACH LINE DOES
1	`# file_client.py - sends a text file over TCP`	Comment: file name and purpose.
2	`import socket`	Imports the `socket` module.
3	`import os`	Imports `os` for `os.path.basename()`.
4		
5	`HOST = "127.0.0.1"`	Server IP.
6	`PORT = 7000`	Server port, 7000.
7	`FILE = "sample.txt"`	Name of the text file to send. It must sit in the same folder as this script.
8		
9	`with open(FILE, "r", encoding="utf-8") as f:`	Opens the file for reading ("r") as UTF-8 text. Doing this first means a missing file fails before we connect.
10	`    content = f.read()`	Reads the whole file into one string.
11		
12	`client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)`	Creates a TCP socket.
13	`client.connect((HOST, PORT))`	Connects to the server.
14		
15	`client.sendall(os.path.basename(FILE).encode())`	Sends only the file's name (no folder path), as bytes.
16	`client.recv(2)`	Waits for the server's 2-byte "OK" before sending the content, keeping name and content separate.
17		
18	`client.sendall(content.encode("utf-8"))`	Encodes the text as UTF-8 bytes and sends the entire file.
19	`client.shutdown(socket.SHUT_WR)`	**Half-close**: tells the server "I'm done sending" (TCP FIN). The server's `recv()` then returns `b""` and its loop ends.
20	`print("Sent", FILE, "to the server.")`	Confirmation message.
21	`client.close()`	Closes the socket completely.

Python 3 · UTF-8 · LF · Spaces: 4 21 lines

Run it

TERMINAL 1
```
python file_server.py
```
TERMINAL 2
```
python file_client.py
```

Put sample.txt in the client's folder. The copy appears as received_sample.txt in the server's folder.

sample.txt (used for testing)
```
Computer Networks Lab - Assignment 3
This file travels from the client to the server over TCP.
TCP guarantees the bytes arrive complete and in order.
Line 4: the end.
```

Sample output

Terminal 1 - server
```
$ python file_server.py
File server waiting on port 7000
Connected to ('127.0.0.1', 39748)
Saved received_sample.txt - 167 bytes
```

Terminal 2 - client
```
$ python file_client.py
Sent sample.txt to the server.
```

Viva guide · Assignment 3

Questions run from definitions to concepts to this program's code. Answers are written the way you would say them in a viva.

LEVEL 1 · BASICS Definitions you should know cold

Q1 What is file transfer? Name a standard protocol for it.

Copying a file from one host to another over a network. FTP is the classic protocol (port 21 for control, 20 for data); SFTP (over SSH, port 22) and HTTP downloads are also common.

Q2 Why use TCP for file transfer?

A file must arrive complete, uncorrupted and in order. TCP retransmits lost segments and puts data back in order; UDP does neither.

Q3 Text file vs binary file?

A text file stores characters using an encoding such as ASCII or UTF-8, organised in lines. A binary file stores arbitrary bytes (images, audio, executables) that are not meant to be read as text.

Q4 What is UTF-8?

A character encoding: English letters take 1 byte, other scripts 2 to 4 bytes. It is compatible with ASCII and is Python's default for `encode()`/`decode()`.

LEVEL 2 · CONCEPTS How and why it works

Q5 Why does the server send "OK" after receiving the file name?

TCP is a byte stream with no message boundaries. If the client sent the name and the content back to back, the server's first `recv()` could get both mixed together. Waiting for OK keeps them separate. Alternatives: a fixed-size header, a length prefix, or a delimiter.

Q6 Why receive in a loop instead of a single recv()?

The file may be larger than the buffer, and TCP can deliver it in several pieces anyway. We keep reading until `recv()` returns `b""`, which signals the end.

Q7 What is a half-close?

`shutdown(socket.SHUT_WR)` closes only our sending direction: TCP sends FIN so the other side sees end-of-data, but we could still receive.

Q8 shutdown() vs close()?

`shutdown()` ends one or both directions of the connection immediately and tells the peer. `close()` releases the socket itself; afterwards it can't be used.

Q9 Why does the server use os.path.basename()?

It keeps only the last part of a path ("docs/notes.txt" becomes "notes.txt"). This stops a client from sending a name like "../../secret" to write files outside the server's folder (a path traversal attack).

LEVEL 3 · CODE WALKTHROUGH Questions about this program

Q10 Why does the client read the file before connecting?

If the file is missing, the error happens before any connection is made, so the server isn't left waiting for data that never comes.

Q11 What does `with open(...) as f` do?

It is a context manager: the file is opened, and closed automatically when the block ends, even if an error occurs.

Q12 Why collect bytes in `data` and decode once at the end?

A UTF-8 character can take several bytes. Decoding each 1024-byte chunk separately could split a character across two chunks and raise UnicodeDecodeError. Decoding the complete data avoids that.

Q13 What does client.recv(2) do?

It waits for the server's 2-byte "OK" acknowledgement, keeping the two programs in step.

Q14 Where is the received file stored?

In the folder the server program is run from, named received_ plus the original name, e.g. received_sample.txt.

Q15 How can you check the file arrived correctly?

Compare the byte counts, open both files, or compare them with `fc` (Windows) / `diff` (Linux), or compare hashes from Python's `hashlib`.

ASSIGNMENT 04

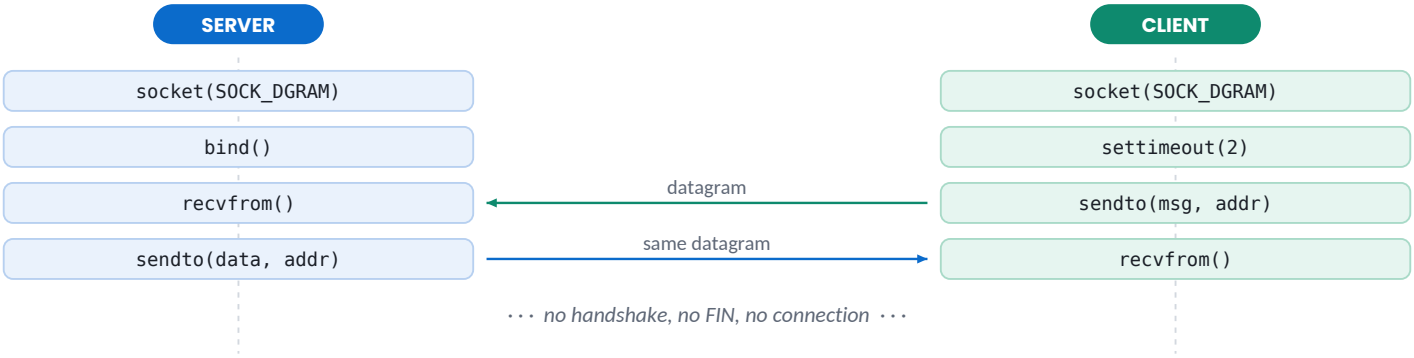
UDP Echo Server-Client Application

Aim. Build a UDP server that returns every datagram it receives to the sender, unchanged.

UDP | Port 8000 | SOCK_DGRAM | udp_echo_server.py | udp_echo_client.py

How it works

There is no connection in UDP: no `listen()`, `accept()` or `connect()`. The server binds to a port and loops on `recvfrom()`, which returns the data **and the sender's address**; `sendto()` uses that address to reply. Because a datagram can be lost, the client sets a 2-second timeout so it never waits forever.



Code with line-by-line explanation

udp_echo_server.py — Python 3

```
1  # udp_echo_server.py - UDP echo server
2  import socket
3
4  HOST = "127.0.0.1"
5  PORT = 8000
6
7  server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
8  server.bind((HOST, PORT))
9  print("UDP echo server running on port", PORT)
10
11 try:
12     while True:
13         data, addr = server.recvfrom(1024)
14         print("From", addr, ":", data.decode())
15         server.sendto(data, addr)
16 except KeyboardInterrupt:
17     print("\nServer shutting down.")
18 finally:
19     server.close()
```

Python 3 · UTF-8 · LF · Spaces: 4 — 19 lines

WHAT EACH LINE DOES
Comment: file name and purpose.
Imports the `socket` module.
Localhost.
Port 8000. UDP and TCP have separate port number spaces.
`SOCK_DGRAM` = **UDP** socket: connectionless; it sends independent datagrams.
Binds to 127.0.0.1:8000 so datagrams sent there reach this program. UDP has no `listen()` or `accept()`.
Status message.
`try` block for a clean Ctrl+C.
Serve forever; this one socket handles every client.
Waits for a datagram. Returns the data (bytes) and the **sender's address**, our only record of who sent it.
Logs who sent what.
Echoes the same bytes back to the exact address they came from.
Runs on Ctrl+C.
Shutdown message.
Always runs…
…closing the socket and freeing the port.

py udp_echo_client.py × Python 3

```python
# udp_echo_client.py - UDP echo client
import socket

HOST = "127.0.0.1"
PORT = 8000

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.settimeout(2)


while True:
    msg = input("Enter message (exit to quit): ")
    if msg.lower() == "exit":
        break
    client.sendto(msg.encode(), (HOST, PORT))
    try:
        data, server_addr = client.recvfrom(1024)
        print("Echo from server:", data.decode())
    except (socket.timeout, ConnectionError):


        print("No reply - the datagram may be lost.")

client.close()
```

Python 3 · UTF-8 · LF · Spaces: 4 21 lines

WHAT EACH LINE DOES
Comment: file name and purpose.
Imports the `socket` module.
Server address.
Server port, 8000.
Creates a UDP socket. No `connect()` needed: there is no connection.
`recvfrom()` will wait at most 2 seconds. Vital in UDP: a lost datagram would otherwise block the client forever.
Loop until the user types "exit".
Reads a message from the keyboard.
Quit command?
Leave the loop.
Sends one datagram to the server's (IP, port). Every `sendto()` names its destination.
`try`: the reply may never arrive.
Waits for the echoed datagram; `server_addr` holds who replied (not used further).
Prints the echo.
`socket.timeout`: no reply in 2 s. `ConnectionError`: on Windows, sending to a port with no server makes the next `recvfrom()` raise ConnectionResetError (ICMP "port unreachable").
Reports the probable loss instead of crashing.
Releases the socket.

Run it

TERMINAL 1
```
python udp_echo_server.py
```
TERMINAL 2
```
python udp_echo_client.py
```

Try running the client with the server stopped: after 2 seconds it reports a lost datagram instead of hanging.

Sample output (text in amber is what you type)

Terminal 1 - server

```
$ python udp_echo_server.py
UDP echo server running on port 8000
From ('127.0.0.1', 47655) : Hello UDP
From ('127.0.0.1', 47655) : No connection needed
^C
Server shutting down.
```

Terminal 2 - client

```
$ python udp_echo_client.py
Enter message (exit to quit): Hello UDP
Echo from server: Hello UDP
Enter message (exit to quit): No connection needed
Echo from server: No connection needed
Enter message (exit to quit): exit

(server stopped)
Enter message (exit to quit): anyone there?
No reply - the datagram may be lost.
```

Viva guide · Assignment 4

Questions run from definitions to concepts to this program's code. Answers are written the way you would say them in a viva.

LEVEL 1 · BASICS Definitions you should know cold

Q1 What is UDP?

User Datagram Protocol: a **connectionless, unreliable**, message-oriented transport-layer protocol. No handshake, no acknowledgements, no retransmission and no ordering.

Q2 Give the main differences between TCP and UDP.

TCP: connection-oriented, reliable, ordered, byte stream, 20-60 byte header, slower. UDP: connectionless, unreliable, unordered, keeps message boundaries, 8-byte header, faster and lighter.

Q3 What is a datagram?

An independent, self-contained message that carries its own destination address. Each datagram is handled separately; the network doesn't link it to any other.

Q4 Where is UDP used?

DNS (53), DHCP (67/68), SNMP (161), TFTP (69), VoIP, live video, online games, and QUIC (HTTP/3): places where speed matters more than guaranteed delivery, or where the application handles reliability itself.

Q5 What fields are in a UDP header?

Source port, destination port, length and checksum: 2 bytes each, 8 bytes in total.

LEVEL 2 · CONCEPTS How and why it works

Q6 Why is UDP faster than TCP?

No connection setup or teardown, no acknowledgements or retransmissions, no congestion-control waiting, and a smaller header.

Q7 Does UDP preserve message boundaries?

Yes. One `sendto()` produces one datagram, which arrives as one `recvfrom()`. This is the opposite of TCP's byte stream.

Q8 What happens if a UDP datagram is lost?

Nothing at the UDP level: nobody notices and nothing is resent. The application has to handle it; here, the client's timeout reports the loss.

Q9 What is the largest UDP datagram?

The length field is 16 bits, so 65,535 bytes in total, of which 65,507 bytes are data over IPv4. In practice data is kept below about 1,472 bytes to avoid IP fragmentation on Ethernet.

LEVEL 3 · CODE WALKTHROUGH Questions about this program

Q10 Why are there no listen(), accept() or connect() calls?

Those exist to set up connections, and UDP has none. The server simply binds and receives from anyone; the client simply sends to an address.

Q11 sendto() vs send(); recvfrom() vs recv()?

`sendto()` takes the destination address on every call; `recvfrom()` returns the data and the sender's address. They're needed because no connected peer is remembered.

Q12 Why does the server need `addr` from recvfrom()?

It is the only record of who sent the datagram, so `sendto(data, addr)` uses it to send the echo back to the right client.

Q13 Why does the client call settimeout(2)?

Without it, a lost datagram would make `recvfrom()` block forever. With it, `recvfrom()` raises `socket.timeout` after 2 seconds and the client carries on.

Q14 Why does the client also catch ConnectionError?

On Windows, sending to a port where no server is listening brings back an ICMP Port Unreachable message, and the next `recvfrom()` raises ConnectionResetError (WinError 10054). Catching it keeps the client running.

Q15 How does one UDP socket serve many clients without threads?

Every datagram carries its sender's address, so the single loop can reply to whoever sent each datagram. There are no per-client connections to manage.

ASSIGNMENT 05

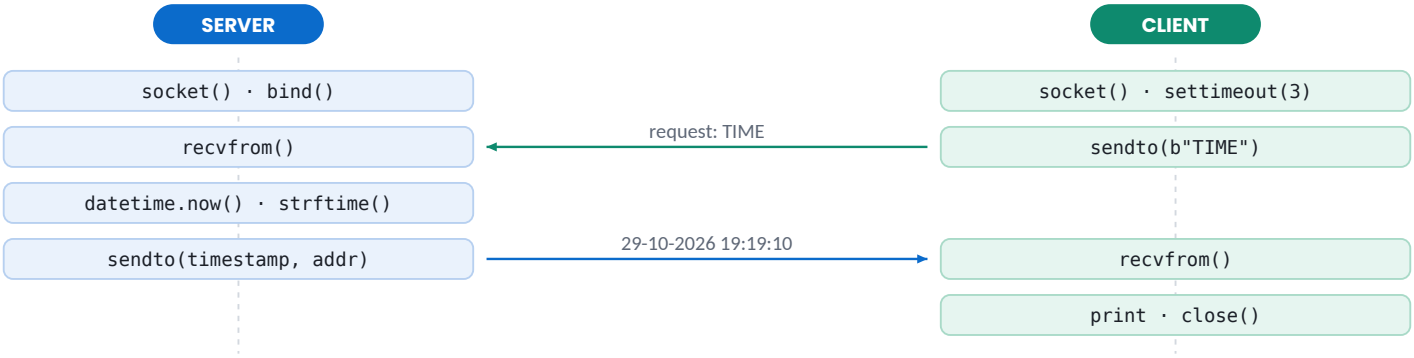
UDP Timestamp Application

Aim. Let a client request the current date and time from a server, which replies with a timestamp over UDP.

UDP | Port 9000 | Request-response | time_server.py | time_client.py

How it works

The client sends a one-word request, **TIME**, in a single datagram. The server checks the request, reads its clock with `datetime.now()`, formats it as day-month-year and 24-hour time, and sends it back to the address the request came from. Anything other than TIME gets an error reply.



Code with line-by-line explanation

py time_server.py × Python 3

		WHAT EACH LINE DOES
1	`# time_server.py - UDP timestamp server`	Comment: file name and purpose.
2	`import socket`	Imports the `socket` module.
3	`from datetime import datetime`	Imports the `datetime` class to read the system clock.
4		
5	`HOST = "127.0.0.1"`	Localhost.
6	`PORT = 9000`	Port 9000 for the timestamp service.
7		
8	`server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)`	Creates a **UDP** socket. A one-question, one-answer service like this suits UDP.
9	`server.bind((HOST, PORT))`	Binds to 127.0.0.1:9000 so requests sent there reach this program.
10	`print("Timestamp server running on port", PORT)`	Status message.
11		
12	`try:`	`try` block for a clean Ctrl+C.
13	`    while True:`	Serve requests forever, from any number of clients.
14	`        data, addr = server.recvfrom(1024)`	Waits for a request datagram; `addr` tells us where to send the answer.
15	`        request = data.decode().strip().upper()`	Bytes to string, `strip()` removes spaces and newlines, `upper()` makes "time" and "TIME" equal.
16	`        if request == "TIME":`	Only the "TIME" command is accepted…
17	`            now = datetime.now()`	…read the current local date and time from the server's clock.
18	`            reply = now.strftime("%d-%m-%Y %H:%M:%S")`	Formats it as text: `%d` day, `%m` month, `%Y` year, `%H:%M:%S` 24-hour time, e.g. 29-10-2026 19:19:10.

py time_server.py × Python 3

```
        else:
            reply = "Invalid request. Send TIME."
        server.sendto(reply.encode(), addr)

        print("Sent", repr(reply), "to", addr)

except KeyboardInterrupt:
    print("\nServer shutting down.")
finally:
    server.close()
```

Python 3 · UTF-8 · LF · Spaces: 4 26 lines

WHAT EACH LINE DOES
Any other request...
...gets an error message instead of the time.
Encodes the reply and sends it to the requesting client's address.
Logs what was sent. `repr()` shows the string in quotes, so spaces are visible.
Runs on Ctrl+C.
Shutdown message.
Always runs...
...closing the socket.

py time_client.py × Python 3

```
# time_client.py - asks the server for the time
import socket

HOST = "127.0.0.1"
PORT = 9000

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.settimeout(3)

client.sendto(b"TIME", (HOST, PORT))

print("Request sent to server.")

try:
    data, server_addr = client.recvfrom(1024)
    print("Server timestamp:", data.decode())
except (socket.timeout, ConnectionError):

    print("Server did not respond.")

client.close()
```

Python 3 · UTF-8 · LF · Spaces: 4 19 lines

WHAT EACH LINE DOES
Comment: file name and purpose.
Imports the `socket` module.
Server address.
Server port, 9000.
Creates a UDP socket.
Wait at most 3 seconds for the answer.
Sends the request "TIME" as a bytes literal in a single datagram to the server.
Confirmation that the request left.
The reply might be lost, so receiving goes inside `try`.
Receives the timestamp datagram and the server's address.
Decodes and prints the server's time.
Timeout after 3 s, or (on Windows) ConnectionError from ICMP "port unreachable": both mean the server isn't answering.
A clear message instead of a crash.
Closes the socket.

Run it

TERMINAL 1	`python time_server.py`
TERMINAL 2	`python time_client.py`

Run the client as many times as you like; each run gets a fresh timestamp.

Sample output

Terminal 1 - server

```
$ python time_server.py
Timestamp server running on port 9000
Sent '29-10-2026 19:19:10' to ('127.0.0.1', 43764)
^C
Server shutting down.
```

Terminal 2 - client

```
$ python time_client.py
Request sent to server.
Server timestamp: 29-10-2026 19:19:10

(server stopped)
$ python time_client.py
Request sent to server.
Server did not respond.
```

Viva guide · Assignment 5

Questions run from definitions to concepts to this program's code. Answers are written the way you would say them in a viva.

LEVEL 1 · BASICS Definitions you should know cold

Q1 What is a timestamp?

A record of a particular moment, either human-readable (29-10-2026 19:19:10) or as a number such as Unix time (1793281750 for that same moment in IST).

Q2 What is Unix (epoch) time?

The number of seconds since 00:00:00 UTC on 1 January 1970. Python returns it with `time.time()`.

Q3 Why do computers need synchronised clocks?

Log ordering, file timestamps, security certificates and logins, distributed databases and financial transactions all depend on machines agreeing on the time.

Q4 Name standard time protocols.

NTP (Network Time Protocol, UDP port 123) synchronises clocks to within milliseconds. The older Daytime protocol (port 13) returns readable time and the Time protocol (port 37) returns seconds since 1900.

LEVEL 2 · CONCEPTS How and why it works

Q5 Why is UDP a good fit for a timestamp service?

The request and the reply each fit in one small datagram, a connection would cost more than the data itself, and if a reply is lost the client can simply ask again. NTP uses UDP for the same reasons.

Q6 What is the request-response pattern?

The client sends a request, the server processes it and sends back a response. HTTP and DNS work the same way.

Q7 Why is the received time already slightly old?

The reply takes time to travel back. NTP corrects for this by measuring the round-trip delay and assuming about half of it was spent on the return trip.

Q8 Local time vs UTC?

`datetime.now()` gives the server's local time (IST is UTC+5:30). Real servers usually send UTC, e.g. `datetime.now(timezone.utc)`, so clients in any time zone interpret it correctly.

LEVEL 3 · CODE WALKTHROUGH Questions about this program

Q9 What does strftime("%d-%m-%Y %H:%M:%S") do?

It formats a datetime as a string: `%d` day, `%m` month, `%Y` four-digit year, `%H` hour (00-23), `%M` minute, `%S` second.

Q10 Why .strip().upper() on the request?

`strip()` removes extra spaces or newlines and `upper()` makes the check case-insensitive, so "time", "Time " and "TIME" are all accepted.

Q11 What happens if the client sends something other than TIME?

The server replies "Invalid request. Send TIME." This is simple input validation: a minimal protocol with one valid command.

Q12 What is b"TIME"?

A bytes literal. It is already bytes, so it can be sent without calling `encode()`.

Q13 What if the server isn't running?

The client waits up to 3 seconds and prints "Server did not respond." On Windows the ConnectionError may arrive sooner.

Q14 How would you send epoch time instead?

`import time` and reply with `str(time.time())` (or `int(time.time())` for whole seconds).

Q15 How could the client measure round-trip time?

Record `time.perf_counter()` just before `sendto()` and again just after `recvfrom()`; the difference is the RTT.

ASSIGNMENT 06

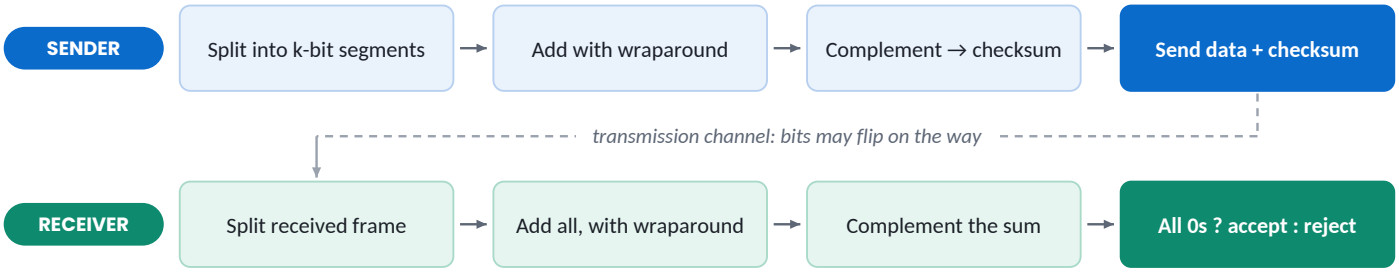
Checksum Error Detection

Aim. Generate a checksum for a data sequence at the sender and use it at the receiver to detect transmission errors.

Error detection | 1's complement | Single program | checksum.py

How it works

The data is cut into k-bit segments that are added with **1's complement arithmetic** (any carry out of the leftmost bit wraps around and is added at the right). The sender complements this sum to get the checksum and sends data + checksum. The receiver adds every received segment, checksum included, and complements again: all zeros means no error was detected.



Worked example

Data **10011001 11100010 00100100 10000100**, segment size k = 8. This is exactly what the program computes (see Run 1 below).

Step	Calculation	Decimal	Binary (k = 8)
Segment 1	start with 10011001	153	10011001
+ Segment 2 (11100010)	153 + 226 = 379 ≥ 256, wrap: 379 − 256 + 1	124	01111100
+ Segment 3 (00100100)	124 + 36 = 160	160	10100000
+ Segment 4 (10000100)	160 + 132 = 292 ≥ 256, wrap: 292 − 256 + 1	37	00100101
Checksum	complement: 255 − 37	**218**	11011010
Receiver: data sum	same four additions as above	37	00100101
+ Checksum	37 + 218 = 255	255	11111111
Complement	255 − 255 = 0: all zeros, so accept	**0**	00000000

Code with line-by-line explanation

py checksum.py × Python 3

		WHAT EACH LINE DOES
1	`# checksum.py - checksum error detection`	Comment: file name and purpose.
2	`def split(bits, k):`	Defines `split()`: cuts the bit string into segments of k bits each.
3	`    if len(bits) % k:`	If the length isn't a multiple of k (a non-zero remainder counts as True)…
4	`        bits = "0" * (k - len(bits) % k) + bits`	…add leading zeros so it divides evenly. Leading zeros don't change the value.
5	`    return [bits[i:i + k] for i in range(0, len(bits), k)]`	List comprehension: slices of k characters starting at 0, k, 2k… e.g. ["10011001", "11100010", …].
6		
7	`def add_wrap(a, b, k):`	Defines `add_wrap()`: adds two k-bit numbers using **1's complement** arithmetic.
8	`    s = a + b`	Ordinary integer addition.
9	`    if s >= 2 ** k:`	If the sum needs more than k bits (sum ≥ 2^k), a carry came out of the leftmost bit…
10	`        s = s - 2 ** k + 1`	…remove that carry (-2^k) and add it back at the right end (+1). This is the **wraparound** carry.
11	`    return s`	Returns the k-bit sum.
12		
13	`def checksum(segments, k):`	Defines `checksum()`. The same function serves the sender and the receiver.
14	`    total = 0`	Running total starts at 0.
15	`    for seg in segments:`	Visit every segment in order.
16	`        total = add_wrap(total, int(seg, 2), k)`	`int(seg, 2)` turns a binary string into an integer; add it to the total with wraparound.
17	`    comp = (2 ** k - 1) - total`	**1's complement** of the sum. $2^k - 1$ is all 1s in k bits, so subtracting from it flips every bit.
18	`    return format(comp, "0" + str(k) + "b")`	Back to a binary string padded to k digits, e.g. format(218, "08b") gives "11011010".
19		
20	`# ---------------- Sender ----------------`	Comment marking the sender part.
21	`data = input("Enter data bits: ").strip()`	Reads the data bits; `strip()` removes stray spaces.
22	`k = int(input("Enter segment size k: "))`	Reads the segment size (e.g. 8 or 16) and converts it to an int.
23	`segs = split(data, k)`	Splits the data into k-bit segments.
24	`cs = checksum(segs, k)`	Sender computes the checksum.
25	`frame = "".join(segs) + cs`	The frame actually sent: all data segments followed by the checksum.
26	`print("Segments        :", " ".join(segs))`	Shows the segments, separated by spaces.
27	`print("Checksum        :", cs)`	Shows the checksum.
28	`print("Transmitted     :", frame)`	Shows the transmitted frame.
29		
30	`# --------------- Receiver ---------------`	Comment marking the receiver part.
31	`recv = input("Received data (Enter = same): ")`	Asks for the frame as it arrived. Change any bit here to simulate a transmission error.
32	`recv = recv.strip() or frame`	Pressing Enter gives an empty string (False), so `or` falls back to the original frame: an error-free transmission.
33	`result = checksum(split(recv, k), k)`	Receiver adds **all** segments (data + checksum) with wraparound, then complements the sum.
34	`print("Receiver result :", result)`	Shows the receiver's result.
35	`if int(result, 2) == 0:`	A result of all zeros means the data passed the check…
36	`    print("No error detected. Data accepted.")`	…so it is accepted.
37	`else:`	Otherwise…
38	`    print("Error detected! Data rejected.")`	…an error occurred and the data is discarded.

Python 3 · UTF-8 · LF · Spaces: 4 38 lines

Run it

TERMINAL

```
python checksum.py
```

Press Enter at the 'Received data' prompt for an error-free run, or type the frame with one bit changed.

Sample output (text in **amber** is what you type)

Run 1 - no error

```
$ python checksum.py
Enter data bits: 10011001111000100010010010000100
Enter segment size k: 8
Segments        : 10011001 11100010 00100100 10000100
Checksum        : 11011010
Transmitted     : 10011001111000100010010010000100 11011010
Received data (Enter = same): ⏎
Receiver result : 00000000
No error detected. Data accepted.
```

Run 2 - one bit flipped in transit

```
$ python checksum.py
Enter data bits: 10011001111000100010010010000100
Enter segment size k: 8
Segments        : 10011001 11100010 00100100 10000100
Checksum        : 11011010
Transmitted     : 10011001111000100010010010000100 11011010
Received data (Enter = same): 10011001111000100010010110000100
Receiver result : 11011001
Error detected! Data rejected.
```

Viva guide · Assignment 6

Questions run from definitions to concepts to this program's code. Answers are written the way you would say them in a viva.

LEVEL 1 · BASICS Definitions you should know cold

Q1 What is a transmission error?

One or more bits changing (0 to 1 or 1 to 0) on the way, because of noise, interference or signal attenuation.

Q2 What types of errors are there?

A **single-bit error** changes one bit. A **burst error** changes two or more bits; its length is counted from the first corrupted bit to the last.

Q3 Error detection vs error correction?

Detection only finds out that an error happened; the data is then discarded and resent (ARQ). Correction also locates and fixes the error, e.g. Hamming code or forward error correction.

Q4 What is redundancy?

Extra bits added to the data purely for error control. A checksum and a CRC are both redundant bits.

Q5 What is a checksum?

A redundancy value found by adding the data's segments with 1's complement arithmetic and complementing the sum. It is sent with the data, and the receiver repeats the calculation to check it.

LEVEL 2 · CONCEPTS How and why it works

Q6 List the steps at the sender.

1) Divide the data into k-bit segments. 2) Add them with 1's complement arithmetic (wrap carries around). 3) Complement the sum: this is the checksum. 4) Send the data followed by the checksum.

Q7 List the steps at the receiver.

1) Divide the received frame into k-bit segments, checksum included. 2) Add them all with wraparound. 3) Complement the sum. 4) All zeros: accept. Anything else: reject.

Q8 What is wraparound (end-around carry)?

In 1's complement addition, a carry out of the leftmost bit is not thrown away; it is added back to the rightmost bit, so the sum always stays k bits wide.

Q9 Why is the sum complemented?

So that at the receiver, data + checksum adds up to all 1s, and its complement is all 0s. Checking for zero is quick and simple.

Q10 Where is a checksum used in real networks?

The 16-bit Internet checksum (RFC 1071) protects the IPv4 header and the TCP and UDP segments.

Q11 What are its limitations?

Addition doesn't care about order, so swapped segments go undetected. Errors that cancel out (one segment increases by the same amount another decreases) are also missed. CRC is far stronger.

LEVEL 3 · CODE WALKTHROUGH Questions about this program

Q12 Why is one checksum() function used at both ends?

Sender: complement of the sum of the data. Receiver: complement of the sum of the data plus the checksum. Both are "complement of a sum", so the same function serves both.

Q13 Explain `s - 2 ** k + 1`.

If the sum is at least 2^k, a carry came out of bit k. Subtracting 2^k removes that carry; adding 1 puts it back at the least significant bit.

Q14 What do int(seg, 2) and format(x, "08b") do?

`int(seg, 2)` reads a binary string as a number ("1010" becomes 10). `format(x, "08b")` writes a number as an 8-digit binary string with leading zeros (10 becomes "00001010").

Q15 Why pad with zeros on the left?

So the length becomes a multiple of k. Leading zeros don't change the value of the first segment, so the result is still correct.

Q16 Work out the textbook example.

Data 10011001 11100010 00100100 10000100 with k = 8. Running sum: 153, then 153 + 226 = 379, wrapped to 124; 124 + 36 = 160; 160 + 132 = 292, wrapped to 37. Complement: 255 − 37 = 218 = **11011010**, which the program prints.

ASSIGNMENT 07

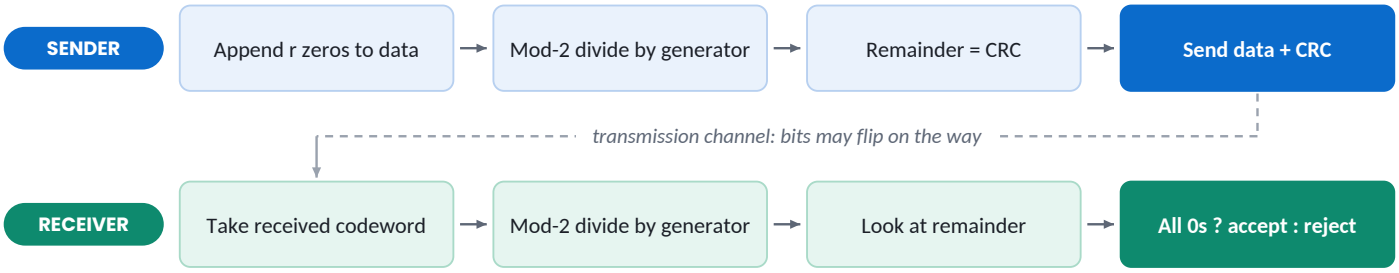
Cyclic Redundancy Check (CRC)

Aim. Generate a CRC with modulo-2 division at the sender and check the received codeword for errors.

Error detection | Modulo-2 division | Single program | crc.py

How it works

Sender and receiver agree on a **generator** of n bits (degree r = n − 1). The sender appends r zeros to the data and divides by the generator using XOR in place of subtraction; the r-bit remainder is the CRC, which replaces the zeros. The receiver divides the whole codeword by the same generator: a remainder of all zeros means no error was detected.



Worked example: modulo-2 division

Data **100100**, generator **1101** ($x^3 + x^2 + 1$), so r = 3. Each row is one pass of the outer `for i` loop in `mod2_remainder()`. Blue bits were just XOR-ed; grey bits are finished (always 0).

i	Working bits	What happens (same steps as the code)
	100100**000**	Start: data 100100 followed by r = 3 zeros (highlighted).
0	**0100**00000	Leading bit is 1, so XOR the generator under positions 0-3: 1001 XOR 1101 = **0100**
1	0**0101**0000	Leading bit is 1, so XOR the generator under positions 1-4: 1000 XOR 1101 = **0101**
2	00**0111**000	Leading bit is 1, so XOR the generator under positions 2-5: 1010 XOR 1101 = **0111**
3	000**0011**00	Leading bit is 1, so XOR the generator under positions 3-6: 1110 XOR 1101 = **0011**
4	00000**0**1100	Leading bit is 0: quotient bit 0, nothing changes.
5	00000**0001**	Leading bit is 1, so XOR the generator under positions 5-8: 1100 XOR 1101 = **0001**
	000000**001**	Remainder = last 3 bits = **001**. Codeword = 100100 + 001 = **100100001**

Code with line-by-line explanation

py crc.py × Python 3

		WHAT EACH LINE DOES
1	`# crc.py - Cyclic Redundancy Check`	Comment: file name and purpose.
2	`def xor(a, b):`	Defines `xor()` for one bit. In modulo-2 arithmetic, subtraction is XOR.
3	`    return "0" if a == b else "1"`	Same bits give "0", different bits give "1".
4		
5	`def mod2_remainder(bits, gen):`	Defines `mod2_remainder()`: binary long division using XOR instead of subtraction. Returns the remainder.
6	`    n = len(gen)`	n = number of bits in the generator (divisor).
7	`    rem = list(bits)`	Turns the string into a list so single bits can be changed in place (strings are immutable).
8	`    for i in range(len(bits) - n + 1):`	Slides the divisor along the dividend one position at a time: len(bits) − n + 1 positions.
9	`        if rem[i] == "1":`	Divide only where the leading bit is 1. If it's 0, the quotient bit is 0 and nothing changes.
10	`            for j in range(n):`	For each bit of the generator…
11	`                rem[i + j] = xor(rem[i + j], gen[j])`	…XOR it with the bit underneath, at position i + j.
12	`    return "".join(rem[len(bits) - (n - 1):])`	After division, the last n − 1 bits are the **remainder**. Join the list back into a string.
13		
14	`# ---------------- Sender ----------------`	Comment marking the sender part.
15	`data = input("Enter data bits: ").strip()`	Reads the data bits.
16	`gen = input("Enter generator bits: ").strip()`	Generator bits, e.g. 1101 = $x^3 + x^2 + 1$. Must start with 1. Sender and receiver use the same one.
17	`r = len(gen) - 1`	r = degree of the generator = number of CRC bits.
18	`padded = data + "0" * r`	Appends r zeros to the data (multiplying it by x^r) to make room for the CRC.
19	`crc = mod2_remainder(padded, gen)`	Divides the padded data by the generator; the remainder is the **CRC**.
20	`codeword = data + crc`	Codeword = original data followed by the CRC (the CRC replaces the zeros).
21	`print("Data + zeros    :", padded)`	Shows data + zeros.
22	`print("CRC (remainder) :", crc)`	Shows the CRC.
23	`print("Transmitted     :", codeword)`	Shows the transmitted codeword.
24		
25	`# --------------- Receiver ---------------`	Comment marking the receiver part.
26	`recv = input("Received data (Enter = same): ")`	Received bits. Change any bit to simulate an error.
27	`recv = recv.strip() or codeword`	Enter alone means use the codeword unchanged.
28	`rem = mod2_remainder(recv, gen)`	Receiver divides the whole received codeword by the same generator.
29	`print("Remainder       :", rem)`	Shows the remainder.
30	`if "1" in rem:`	Any 1 in the remainder means the codeword isn't exactly divisible…
31	`    print("Error detected! Data rejected.")`	…so an error is detected.
32	`else:`	Remainder all zeros…
33	`    print("No error detected. Data accepted.")`	…data accepted.

Python 3 · UTF-8 · LF · Spaces: 4 33 lines

Run it

TERMINAL

```
python crc.py
```

Press Enter at the 'Received data' prompt for an error-free run, or retype the codeword with a bit changed.

Sample output (text in amber is what you type)

Run 1 - no error

```
$ python crc.py
Enter data bits: 100100
Enter generator bits: 1101
Data + zeros    : 100100000
CRC (remainder) : 001
Transmitted     : 100100001
Received data (Enter = same): ⏎
Remainder       : 000
No error detected. Data accepted.
```

Run 2 - one bit flipped in transit

```
$ python crc.py
Enter data bits: 100100
Enter generator bits: 1101
Data + zeros    : 100100000
CRC (remainder) : 001
Transmitted     : 100100001
Received data (Enter = same): 100110001
Remainder       : 111
Error detected! Data rejected.
```

Viva guide · Assignment 7

Questions run from definitions to concepts to this program's code. Answers are written the way you would say them in a viva.

LEVEL 1 · BASICS Definitions you should know cold

Q1 What is CRC?

Cyclic Redundancy Check: an error-detection method based on binary polynomial division. The remainder of the division (the CRC) is appended to the data.

Q2 What is the generator (divisor)?

A bit pattern agreed by sender and receiver that represents a polynomial. 1101 represents $x^3 + x^2 + 1$.

Q3 How is a bit string written as a polynomial?

Each bit is a coefficient of a power of x, starting from the right at x^0. So 100100 is $x^5 + x^2$.

Q4 What is modulo-2 arithmetic?

Binary arithmetic with no carries or borrows. Addition and subtraction are both simply XOR.

Q5 Where is CRC used?

Ethernet frames (CRC-32 in the Frame Check Sequence), Wi-Fi, HDLC, USB, and file formats such as ZIP and PNG.

LEVEL 2 · CONCEPTS How and why it works

Q6 List the steps at the sender.

1) r = number of generator bits − 1. 2) Append r zeros to the data. 3) Divide by the generator with modulo-2 division. 4) The r-bit remainder is the CRC. 5) Send data + CRC.

Q7 List the steps at the receiver.

Divide the received codeword by the same generator. A remainder of all zeros means no error was detected; anything else means an error.

Q8 Why append r zeros before dividing?

It multiplies the data by x^r, shifting it left to make room for the r-bit remainder. Replacing those zeros with the remainder makes the codeword exactly divisible by the generator.

Q9 Why does a zero remainder mean no error?

The codeword was built to be an exact multiple of the generator. Any error pattern that isn't itself a multiple of the generator leaves a non-zero remainder.

Q10 Which errors does CRC detect?

All single-bit errors (generator with at least two terms), all burst errors of length up to r, all errors with an odd number of bits when the generator has (x + 1) as a factor, and most longer bursts.

Q11 CRC vs checksum?

CRC uses polynomial division, is sensitive to bit positions and catches burst errors much better; it is usually done in hardware at the data link layer. A checksum uses addition: simpler and faster in software, but weaker. It is used at the network and transport layers.

Q12 Name some standard generators.

CRC-8, CRC-16 ($x^{16} + x^{15} + x^2 + 1$) and CRC-32, which Ethernet uses (written in hex as 0x04C11DB7).

LEVEL 3 · CODE WALKTHROUGH Questions about this program

Q13 Explain the xor() function.

It follows the XOR truth table on the characters "0" and "1": equal bits give "0", different bits give "1".

Q14 Why convert the bit string to a list?

Python strings are immutable. A list lets us change individual bits in place as the division proceeds.

Q15 Why XOR only when rem[i] is "1"?

In long division, if the leading bit is 0 the quotient bit is 0 and we would XOR with all zeros, which changes nothing. Skipping it gives the same result.

Q16 Why does the outer loop run len(bits) - n + 1 times?

That is the number of positions where the whole generator fits under the dividend.

Q17 What does the last line return?

The final n − 1 bits of the list after division: the remainder, r bits long.

Q18 Work out the textbook example.

Data 100100, generator 1101, so r = 3. Divide 100100000 by 1101: the remainder is **001**, so the codeword is 100100001. The step-by-step table above shows each XOR.

ASSIGNMENT 08

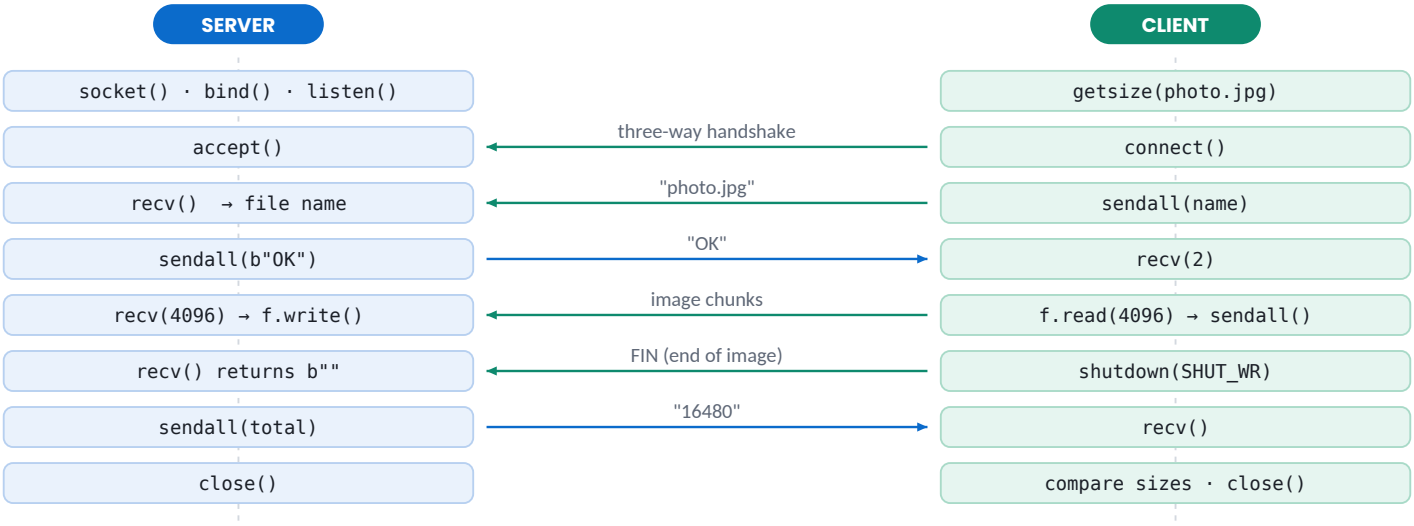
TCP Image File Transfer Application

Aim. Send an image file from the client to the server over TCP and store an exact copy on the server.

TCP | Port 7500 | Binary mode | image_server.py | image_client.py

How it works

Same handshake as the text transfer (name, OK, data, half-close), but everything stays **binary**: the client reads the image with `"rb"` in 4096-byte chunks and the server writes each chunk straight to disk with `"wb"`, never decoding it. After the client half-closes, the server replies with the byte count so the client can confirm the copy is complete.



Code with line-by-line explanation

py image_server.py × Python 3

```python
# image_server.py - receives an image over TCP
import socket
import os

HOST = "127.0.0.1"
PORT = 7500

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen(1)
print("Image server waiting on port", PORT)

conn, addr = server.accept()
print("Connected to", addr)

filename = conn.recv(1024).decode()
conn.sendall(b"OK")
save_as = "received_" + os.path.basename(filename)

total = 0
```

WHAT EACH LINE DOES
Comment: file name and purpose.
Imports the `socket` module.
Imports `os` for `os.path.basename()`.
Localhost.
Port 7500 for the image service.
TCP socket: every byte of an image must arrive, in order, or the picture is corrupted.
Allows a quick restart on the same port.
Binds to 127.0.0.1:7500.
Listens for one client.
Status message.
Waits for the client to connect.
Shows the client's address.
Receives the image's file name and decodes it.
Sends "OK" so the client starts the image only after the name was received on its own.
Output name with any folder path stripped, e.g. received_photo.jpg.
Counter for the bytes received.

py image_server.py × Python 3

```python
with open(save_as, "wb") as f:

    while True:
        chunk = conn.recv(4096)

        if not chunk:

            break
        f.write(chunk)

        total += len(chunk)

print("Saved", save_as, "-", total, "bytes")
conn.sendall(str(total).encode())

conn.close()
server.close()
```

Python 3 · UTF-8 · LF · Spaces: 4 33 lines

WHAT EACH LINE DOES
Opens the output file in **binary write** mode ("wb"): bytes are stored exactly as received, no text decoding.
Receive until the client signals the end.
Reads up to 4096 bytes per call (a common buffer size; fewer calls than 1024).
Empty bytes = the client called `shutdown()`: the whole image is here.
Leave the loop.
Writes this chunk straight to disk, so even a large image never has to fit in memory.
Adds this chunk's size to the running total.
Reports the file name and byte count.
Sends the total back as a confirmation. Possible because the client closed only its *sending* direction.
Closes the client connection.
Closes the listening socket.

py image_client.py × Python 3

```python
# image_client.py - sends an image over TCP
import socket
import os

HOST = "127.0.0.1"
PORT = 7500
FILE = "photo.jpg"


size = os.path.getsize(FILE)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

client.sendall(os.path.basename(FILE).encode())
client.recv(2)


with open(FILE, "rb") as f:

    while True:
        chunk = f.read(4096)
        if not chunk:
            break
        client.sendall(chunk)

client.shutdown(socket.SHUT_WR)

reply = client.recv(1024).decode()
print("Sent", size, "bytes; server got", reply)
client.close()
```

Python 3 · UTF-8 · LF · Spaces: 4 26 lines

WHAT EACH LINE DOES
Comment: file name and purpose.
Imports the `socket` module.
Imports `os` for file size and file name helpers.
Server IP.
Server port, 7500.
Image to send, kept in the same folder as this script. Any format works (.jpg, .png, .bmp).
Size of the image in bytes. Also fails early (FileNotFoundError) if the file is missing, before we connect.
Creates a TCP socket.
Connects to the server.
Sends just the file name, as bytes.
Waits for "OK" so the name and the image data stay separate.
Opens the image in **binary read** mode ("rb"): raw bytes, no text decoding.
Read and send piece by piece.
Reads the next 4096 bytes from the file.
`f.read()` returns empty bytes at end of file…
…so stop.
Sends this chunk completely.
Half-close: signals end of data (FIN) but keeps the socket open to hear the server's reply.
Receives the byte count the server reports.
The two numbers must match for a complete transfer.
Closes the socket.

Run it

TERMINAL 1	`python image_server.py`
TERMINAL 2	`python image_client.py`

Put photo.jpg (or change FILE to any image) in the client's folder. Open received_photo.jpg afterwards: it is identical.

Sample output

Terminal 1 - server

```
$ python image_server.py
Image server waiting on port 7500
Connected to ('127.0.0.1', 33816)
Saved received_photo.jpg - 16480 bytes
```

Terminal 2 - client

```
$ python image_client.py
Sent 16480 bytes; server got 16480
```

Verified during testing: the MD5 hash of photo.jpg and received_photo.jpg is identical (251373b2...c93f), so the copy is byte-for-byte exact.

Viva guide · Assignment 8

Questions run from definitions to concepts to this program's code. Answers are written the way you would say them in a viva.

LEVEL 1 · BASICS Definitions you should know cold

Q1 How is an image stored in a file?

As binary data: a header (format, width, height and so on) followed by pixel data, usually compressed. It is not readable text.

Q2 Text mode vs binary mode in open()?

Text mode ("r", "w") decodes bytes into characters and may convert line endings (\r\n on Windows). Binary mode ("rb", "wb") reads and writes raw bytes untouched. Images must use binary mode.

Q3 What is a file signature (magic number)?

The first few bytes that identify a file's format. JPEG files start with FF D8 FF; PNG files start with 89 50 4E 47.

Q4 What is a MIME type?

A label that tells software what kind of data is being sent, e.g. image/jpeg, image/png, text/plain. HTTP and email use it.

LEVEL 2 · CONCEPTS How and why it works

Q5 Why must image data never be decoded as text?

Image bytes are not valid UTF-8. `decode()` would raise UnicodeDecodeError or change bytes, corrupting the picture.

Q6 Why send the image in 4096-byte chunks?

Images can be many megabytes. Chunks keep memory use small and match typical buffer sizes; 4096 is a common page and buffer size.

Q7 Why TCP and not UDP for images?

A single lost or misplaced byte can corrupt an image. TCP guarantees complete, ordered delivery. Over UDP we would have to add our own numbering and acknowledgements (as TFTP does).

Q8 How does the server know the whole image has arrived?

It reads until `recv()` returns `b""`, which happens when the client half-closes. It then reports the total, and the client compares it with the file size.

Q9 How else could the end of a file be marked?

Send the file size first in a fixed-size header (e.g. `struct.pack("!Q", size)`), then receive exactly that many bytes. This also lets the same connection carry more files.

Q10 How can you verify the copy is identical?

Compute a hash such as MD5 or SHA-256 with `hashlib` on both files and compare. Equal hashes mean identical files. (We did this during testing.)

LEVEL 3 · CODE WALKTHROUGH Questions about this program

Q11 How is this different from the text transfer in assignment 3?

Binary modes "rb"/"wb", no `encode()`/`decode()` of the content, chunks are written straight to disk instead of collected in memory, and the server confirms the byte count.

Q12 How can the client still recv() after shutdown(SHUT_WR)?

A half-close shuts only the sending direction. The receiving direction stays open, so the server's reply still gets through.

Q13 What does `total += len(chunk)` do?

It counts the bytes actually received, which the server reports back as confirmation.

Q14 Why does the client call os.path.getsize() before connecting?

It gets the size for the final comparison, and if the image is missing it fails right away with FileNotFoundError, before the server is involved.

Q15 Can this code send other kinds of files?

Yes. Binary mode works for any file (PDF, MP3, ZIP); the file name keeps the extension.